

LLM Observability & AI Cost-Optimization Platform

Build a web application that measures, visualizes, and optimizes how AI is used across an organization.

Organized by: Aigenthix Technologies Private Limited

Dates: 15/06/2026 - 16/06/2026

1. Overview

As organizations adopt large language models (LLMs) across products and internal tools, a recurring blind spot emerges: teams rarely know how much their AI actually costs, how fast it responds, or where the money is going. This hackathon asks participants to close that gap.

The challenge is to build a web application for LLM observability — a platform that lets a user connect their own APIs, automatically surfaces the AI usage inside them, and renders the resulting KPIs and metrics on a live dashboard. On top of that observability layer, teams will build an AI optimization agent that reads the collected data and recommends concrete, actionable ways to cut LLM cost while keeping the system scalable.

2. The Challenge

Build a working web application that does three things:

- **Integrates any API and instruments it.** Let a user register an API endpoint and route or wrap its LLM traffic so the platform can observe each call.
- **Visualizes AI KPIs in real time.** Render charts and metrics — input/output tokens, response time, throughput, cost, error rates, and more — on an interactive dashboard.
- **Recommends optimizations through an agent.** An AI agent analyzes the collected telemetry, identifies the biggest cost drivers, and proposes specific, scalable changes with projected savings.

Important scoping note: “observe any API” only works when the AI usage is exposed somewhere. Section 4.3 defines the three capture patterns teams may use — read it before designing, as it is the single most common place teams get stuck.

3. Objectives

A strong submission should demonstrate the following outcomes:

- A user can connect at least one LLM-backed API and see its activity reflected on the dashboard within minutes.

- Token usage and cost are tracked accurately per request and aggregated over time, by model, and by endpoint or feature.
- Performance and reliability metrics (latency, throughput, error rate) are visible and update as new calls come in.
- The optimization agent produces recommendations grounded in the team’s own data — not generic advice — with an estimated cost impact.
- The ingestion and storage design would not fall over under realistic load (the “scalable approach”).

4. Functional Requirements

4.1 The Observability Dashboard

The dashboard is the heart of the product. A user should be able to add an API integration, then watch metrics populate as traffic flows through it. Charts should be filterable by time range, model, and endpoint, and should support drilling from an aggregate number down to individual requests (a request log with prompt, completion, tokens, latency, and cost per call).

4.2 Metrics to Track

At minimum, teams should capture and visualize the metrics below. Breadth and clarity here are directly rewarded in judging.

Category	Metrics
Cost & Tokens	Input tokens, output tokens, total tokens per request; cost per request (tokens × model price); cost per user / feature / day; cumulative spend over time.
Performance	Total response time (latency); time-to-first-token (TTFT); tokens per second (throughput); requests per second.
Reliability	Error rate; status-code breakdown; timeout rate; rate-limit (429) rate; retry count.
Usage Shape	Requests over time; model mix; endpoint / feature mix; average prompt and completion length.
Advanced (stretch)	Eval or quality scores; user feedback (thumbs up/down); prompt-cache hit rate; cost attribution by tenant.

4.3 Token & Cost Capture — the critical scoping decision

You cannot point at an arbitrary REST endpoint and magically know its internal LLM token usage — that data has to be exposed somewhere. There are three capture patterns. Teams should state clearly which one(s) they support.

Pattern	How it works
Provider usage fields	When the app calls an LLM provider directly, the response body already contains a usage object with input/output token counts. Trivial to log — works whenever the raw LLM response is visible.
Proxy / gateway	Route LLM traffic through a logging proxy that sits in the middle (the pattern behind tools like Helicone and LiteLLM). The integrated API points its base URL at the team's proxy, which transparently logs tokens, latency, and cost. This is the real answer to “integrate anything” — and the most scalable architecture.
Tokenizer estimation	When only prompt and completion text are visible but no usage field is returned, estimate tokens with the model's tokenizer (e.g. tiktoken for OpenAI-family models). Approximate, but the only option for opaque APIs.

Recommendation: frame the challenge around the proxy/gateway or provider-usage approach. Teams that attempt to read tokens from a fully opaque third-party API without one of these mechanisms will not get accurate numbers.

4.4 The AI Optimization Agent

The agent should not give generic advice. It should read the team's own collected telemetry, find the largest cost drivers, and propose specific changes with an estimated impact. A winning example of output: “Your summarization endpoint costs \$X/day and 90% of calls are short — route it to a smaller model for an estimated 70% saving.”

Cost-reduction levers the agent can reason about and recommend:

- **Model cascading / right-sizing** — send easy queries to a cheaper/smaller model and escalate only the hard ones.
- **Prompt caching & semantic caching** — reuse responses for identical or similar inputs.
- **Context reduction** — trim prompts or use retrieval (RAG) instead of stuffing large context.
- **Batch processing** — move non-realtime workloads to a batch API for a steep discount.
- **Output control** — cap max output tokens and use structured outputs to avoid runaway generations.
- **Distillation / fine-tuning** — replace a large model with a small fine-tuned one for repetitive, narrow tasks.

5. Technical Guidance

5.1 Scalable Architecture

The “scalable approach” the brief asks for comes down to one principle: observability must never slow down the thing it observes. The recommended pattern is an asynchronous pipeline.

- **Non-blocking logging** — capture telemetry on the call path without adding latency; fire-and-forget to a buffer.

- **Buffer / queue** — absorb spikes with a queue or stream (e.g. Redis Streams, Kafka).
- **Time-series / columnar store** — persist events in a store built for analytics (ClickHouse is the de facto choice; Postgres or TimescaleDB is fine within a hackathon).
- **Aggregation + dashboard** — query and roll up the data behind an API that feeds the charts.

Optional bonus: adopting the OpenTelemetry GenAI semantic conventions makes the telemetry portable and is a strong signal of a production-minded design.

5.2 Suggested Tech Stack

Teams are free to use any stack. The options below are starting points, not requirements.

Layer	Suggested options
Frontend / dashboard	React or Next.js with a charting library — Recharts, Chart.js, Tremor, or Plotly.
Backend / API	FastAPI (Python) or Node.js / Express.
Ingestion	A proxy/gateway in front of LLM calls, or an SDK wrapper; OpenTelemetry GenAI conventions for portability.
Transport / buffer	Redis Streams or Kafka for async, spike-tolerant ingestion.
Storage	ClickHouse for analytics scale; Postgres / TimescaleDB for simplicity.
Optimization agent	Any LLM with function/tool calling that can query the metrics store and reason over the data.
Tokenization	tiktoken or the relevant provider tokenizer for estimation when usage fields are absent.

6. Deliverables

- **Working web application** — deployed or runnable locally, demonstrating live ingestion and the dashboard.
- **Source code repository** — with a README covering setup and how to connect an API.
- **Short demo** — a live walkthrough or a 3–5 minute recorded video showing the full flow.
- **Architecture note** — a one-page diagram or description of the ingestion and storage design.
- **Agent samples** — at least two example optimization recommendations produced from real captured data.

7. Judging Criteria

Submissions will be scored against the following rubric (total 100 points).

Criterion	What we're looking for	Weight
Optimization agent quality	Recommendations are specific, grounded in the team's own data, and quantify expected savings.	25
Token & cost accuracy	Token counts and cost are captured correctly via a sound capture pattern.	20
Metrics & dashboard breadth	Range and clarity of metrics; useful filtering and drill-down.	20
Architecture & scalability	Async, non-blocking design that would hold up under realistic load.	15
UX & dashboard design	The dashboard is clear, responsive, and pleasant to use.	10
Integration & documentation	How easily a new API can be connected; quality of the README.	10

8. Suggested Timeline

Adjust to your event length. The split below assumes a standard multi-hour or weekend hackathon.

Phase	Focus	Share
Setup	Pick a capture pattern; stand up ingestion and a store; wire one API end to end.	~25%
Dashboard	Build the core metrics and charts; add filtering and a request log.	~30%
Agent	Connect the agent to the metrics store; produce grounded recommendations.	~25%
Polish & demo	UX cleanup, README, architecture note, and demo prep.	~20%

9. Rules & Submission

- Team size: 2–4 participants per team.
- Use of open-source libraries and APIs is encouraged; all third-party use must be disclosed in the README.
- Pre-built code may be used only if clearly attributed; the core observability and agent logic should be built during the event.
- Submit via email with the github link and readme before 16/06/2026 8:00 PM. .
- Prizes: .

10. Resources & References

Reference products in this space worth exploring for inspiration (not for copying):

- Open-source LLM observability tools such as Langfuse, Helicone, and Arize Phoenix.
- LLM gateways/proxies such as LiteLLM for the proxy-capture pattern.
- The OpenTelemetry GenAI semantic conventions for standardized AI telemetry.